# Caching Strategies

Caching strategies determine how, when, and where data is stored temporarily to improve performance and reduce load on backend systems.

## Common Caching Strategies

### 1. Cache-Aside (Lazy Loading)

The application checks the cache first. If data isn't found (cache miss), it fetches from the database, then stores it in the cache for future requests.

**When to use**: General-purpose caching, read-heavy workloads, Data that doesn't change frequently

**Pros**: Only caches data that's actually requested, simple to implement

**Cons**: Initial requests are slow (cache miss penalty), potential for stale data

### 2. Write-Through

Data is written to both the cache and database simultaneously. The cache always stays in sync with the database.

**When to use:** When data consistency is critical, write-heavy applications

**Pros:** Cache is always up-to-date, no stale data

**Cons:** Higher write latency, wasted cache space for data that's never read

### 3. Write-Behind (Write-Back)

Data is written to the cache immediately, then asynchronously written to the database later.

**When to use:** High write throughput needed, can tolerate some data loss risk

**Pros:** Very fast writes, reduced database load

**Cons: Risk** of data loss if cache fails before database write, complex implementation

### 4. Read-Through

Cache sits between the application and database. On a cache miss, the cache itself loads data from the database.

**When to use:** Simplified application code, centralized caching logic

**Pros:** Application doesn't need to manage cache population

**Cons:** First request is still slow, requires cache provider support

### 5. Refresh-Ahead

Automatically refreshes frequently accessed cache entries before they expire.

**When to use:** Predictable access patterns, critical data that must always be fast

**Pros:** Eliminates cache miss latency for hot data

**Cons:** Can waste resources refreshing unused data, requires prediction logic

## Cache Invalidation Strategies

1. **Time-to-Live (TTL) :** Cache entries expire after a set time period. **Simple and predictable**, but may serve stale data before expiration or waste cache space after data changes.
2. **Event-Based Invalidation :** Cache is cleared or updated when specific events occur (like database updates). **Keeps cache accurate**, but requires event infrastructure and can be complex to implement correctly.
3. **Manual Invalidation :** Application explicitly removes or updates cache entries when data changes. **Full control**, but error-prone and requires careful coordination across the codebase.

## Common Real-World Combos

Most production systems mix strategies:

- **Cache-aside + TTL** (default choice)
- **Write-through + LRU** for strong consistency
- **Cache-aside + Kafka invalidation** for microservices
- **Local in-memory cache + Redis** (2-level cache)

### Classic Caching Pitfalls (interview gold ⭐)

- Cache stampede (many requests on expiry)
- Hot keys
- Over-caching rarely used data
- Forgetting cache eviction
- Treating cache as a source of truth

### Multi-Level Caching

Combining multiple cache layers for optimal performance:

1. **Browser cache**: Static assets, API responses
2. **CDN cache**: Geographically distributed content
3. **Application cache**: In-memory (Redis, Memcached)
4. **Database cache**: Query results, indexes